

函数（参数、递归、闭包、高阶函数）

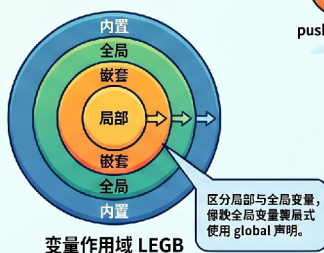
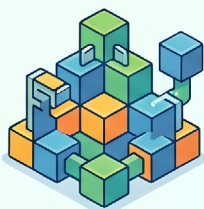
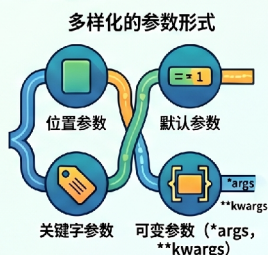


学习目标：

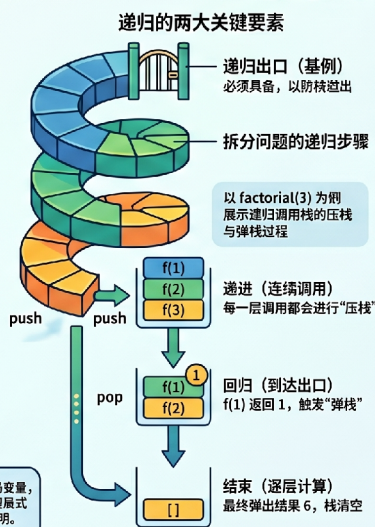
1. 理解“函数”是什么，为什么需要函数。
2. 能够定义自己的函数，使用参数和返回值。
3. 区分局部变量和全局变量。
4. 掌握 lambda 匿名函数及高阶函数（map/filter/sorted）。
5. 理解递归函数的工作原理（压栈/弹栈），能写简单递归。
6. 理解函数的嵌套定义。
7. 理解闭包的概念（三要素）和用途。

Python 函数核心指南：从基础到进阶

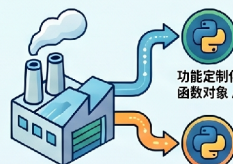
函数基础与参数规则



递归机制与闭包特性



闭包的“背包”效应
内部函数记住外层变量，即便外部函数执行完毕，变量依然存活。



工厂函数应用
利用闭包根据不同参数动态“生产”出功能定制化的函数对象。

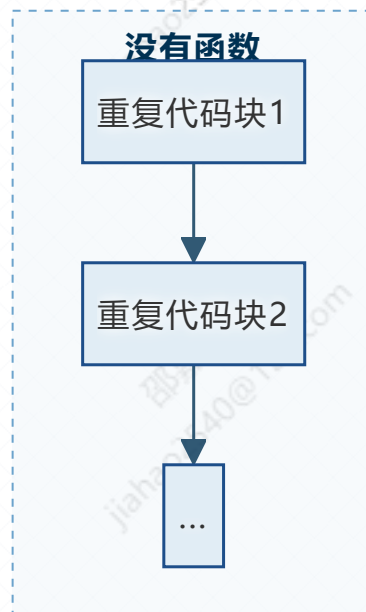
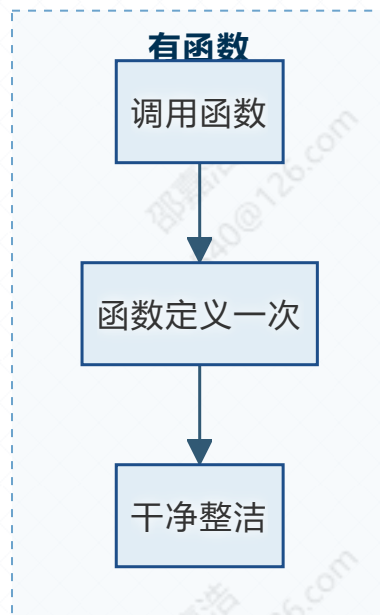
第八章：函数 (Funcation)

1. 为什么需要函数？

生活类比：假设你每天都要做三明治：拿面包 → 放生菜 → 放火腿 → 盖面包。如果没有“函数”，你需要重复写这四行代码很多次。而有了函数，你只需要**定义一次**“做三明治”的步骤，然后每次喊一声“做个三明治”就行了。

在编程中：

- **避免重复：**相同的代码只需要写一次。
- **模块化：**把一个复杂的大问题拆成几个小问题，每个小问题用一个函数解决。
- **易维护：**如果需要修改某个功能，只需要改函数内部，不用满世界找代码。



2. 函数的定义与调用

2.1 基本语法



python

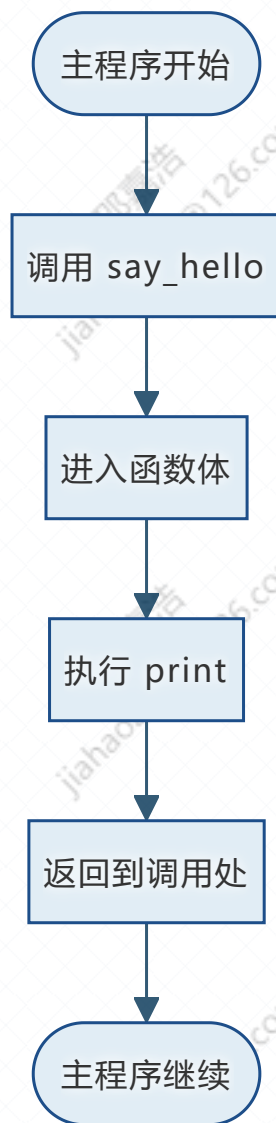
```
1 def 函数名(参数1, 参数2, ...):  
2     """可选的文档字符串（说明函数的作用）"""  
3     函数体代码
```

- **def** 关键字，告诉 Python “我要定义一个函数啦”。
- 函数名命名规则同变量名（小写+下划线）。
- **参数**：函数需要的“原料”。
- **return**：函数产出什么结果。如果没有 **return**，函数返回 **None**。

2.2 最简单的函数：无参数，无返回值

```
python
1  def say_hello():
2      print("Hello, Python!")
3
4  # 调用函数
5  say_hello() # 输出 Hello, Python!
```

流程图：函数调用



2.3 带参数和返回值的函数

```
python
1  def add(a, b):
2      """返回两个数的和"""
3      result = a + b
4      return result
5
6  sum_result = add(5, 3)  # 调用并保存返回值
7  print(sum_result)      # 8
```

`return` 会立即结束函数，并返回后面的值。`return` 后面的代码不会执行。

3. 参数的多种形式

3.1 位置参数（最常用）

调用时传入的值按顺序赋给参数。

```
python
1 def greet(name, greeting):
2     print(f"{greeting}, {name}!")
3
4 greet("小明", "你好")    # 你好, 小明!
5 greet("你好", "小明")    # 小明, 你好!    (顺序错了, 语义不对)
```

3.2 默认参数

给参数指定默认值，调用时可以不传。

```
python
1 def greet(name, greeting="你好"):
2     print(f"{greeting}, {name}!")
3
4 greet("小明")            # 你好, 小明!
5 greet("小红", "Hello")  # Hello, 小红!
```

注意：默认参数必须放在非默认参数后面。`def greet(greeting="你好", name):`
✗ 语法错误。

3.3 关键字参数

调用时明确指定参数名，可以改变顺序。

```
1 greet(name="小华", greeting="Hi")    # Hi, 小华!  
2 greet(greeting="哈喽", name="小丽") # 哈喽, 小丽!
```

3.4 可变参数: `*args` 和 `**kwargs`

- `*args`: 接收任意多个位置参数, 把它们打包成一个元组。

- `**kwargs` : 接收任意多个关键字参数, 打包成一个字典。



名字约定: `*args` 和 `**kwargs` 是 Python 社区的**强约定**。虽然语法上允许写成 `*numbers` 或 `**options`, 但**强烈建议永远不要修改这两个名字**。任何有经验的 Python 开发者看到 `*args` 和 `**kwargs` 会立即明白含义; 如果改成别的名字 (如 `*params`), 会造成困惑和维护困难。



python

```
1  def sum_all(*args):
2      """计算所有传入位置参数的和"""
3      total = 0
4      for n in args:
5          total += n
6      return total
7
8  print(sum_all(1, 2, 3))          # 6
9  print(sum_all(10, 20, 30, 40)) # 100
10
11 def show_info(**kwargs):
12     """打印所有传入的关键字参数"""
13     for key, value in kwargs.items():
14         print(f"{key}: {value}")
15
16 show_info(name="张三", age=20, city="北京")
17 # 输出:
18 # name: 张三
19 # age: 20
20 # city: 北京
```

4. 函数的返回值

- 可以返回任意类型 (数字、字符串、列表、字典……)。

- 可以返回**多个值**（实际上是以元组形式返回）

```
python
1  def get_min_max(numbers):
2      return min(numbers), max(numbers)
3
4  result = get_min_max([5, 2, 9, 1])
5  print(result)          # (1, 9)
6  min_val, max_val = get_min_max([5, 2, 9, 1])
7  print(min_val, max_val) # 1 9
```

5. 变量作用域：局部 vs 全局

5.1 局部变量

在**函数内部**定义的变量，只能在函数内部使用。

```
python
1  def func():
2      x = 10    # 局部变量
3      print(x)
4
5  func()
6  print(x)     # 错误! NameError: name 'x' is not defined
```

5.2 全局变量

在**函数外部**定义的变量，可以在函数内部**读取**，但不能直接**修改**（除非用 `global`）。

```
python
1  y = 20    # 全局变量
2
3  def show():
```

```

4      print(y)    # 可以读取，输出 20
5
6      def modify():
7          y = 100    # 这会在函数内部创建一个新的局部变量 y，不影响全局 y
8          print("函数内部:", y)
9
10     show()
11     modify()      # 函数内部: 100
12     print(y)      # 20    全局变量没变

```

5.3 global 关键字

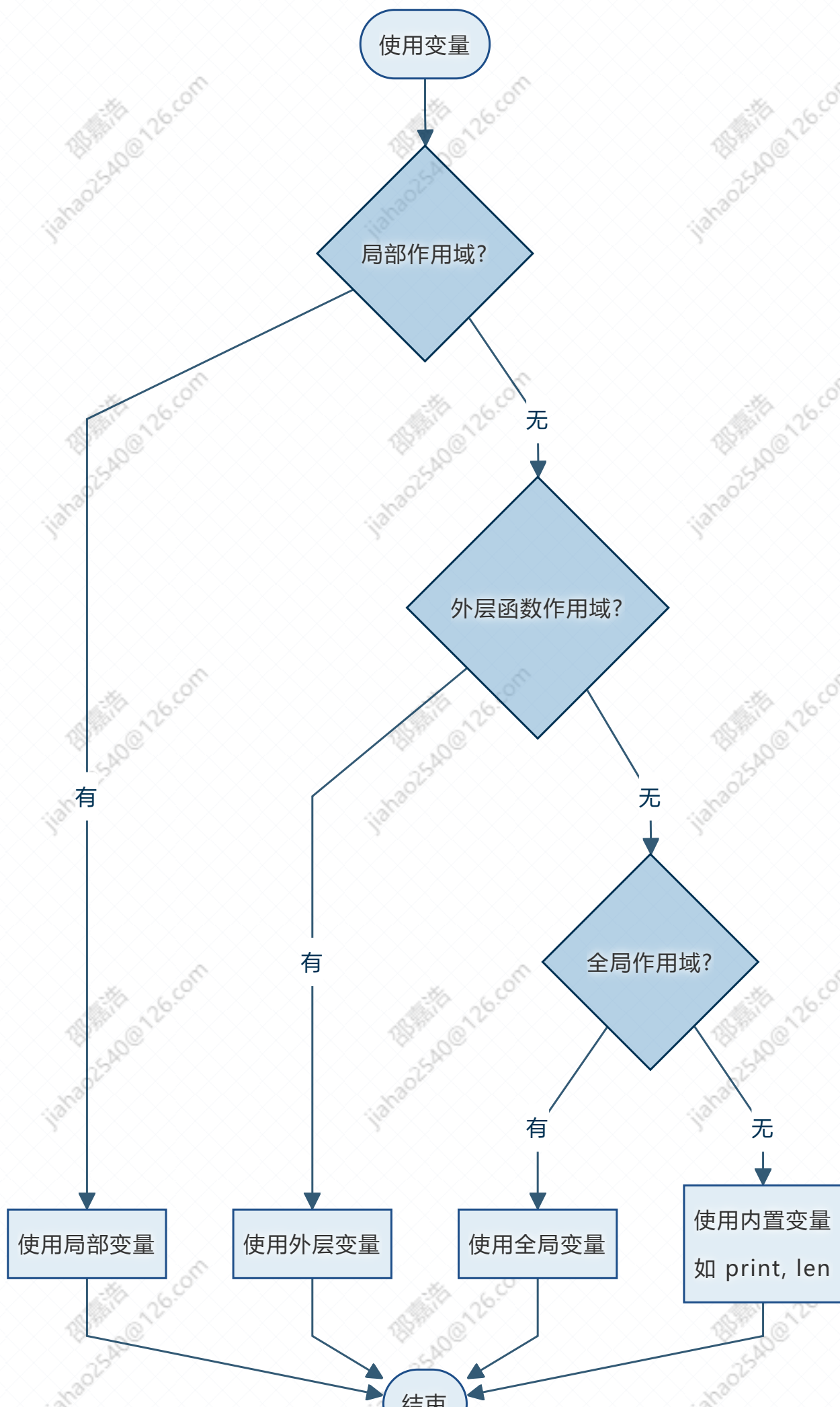
如果需要修改全局变量，用 `global` 声明。

```

python
1      count = 0
2
3      def increment():
4          global count
5          count += 1
6
7      increment()
8      print(count)    # 1

```

流程图：作用域查找顺序（LEGB规则）





尽量少用 `global`，因为它会让代码逻辑变乱。最好通过参数传递和返回值来交互。

6. Lambda 匿名函数

定义：没有名字的、一行就能写完的简单函数。适用于作为参数传给其他函数。

语法： `lambda 参数: 表达式`

```
python
1  # 普通函数
2  def square(x):
3      return x ** 2
4
5  # 等价的lambda
6  square_lambda = lambda x: x ** 2
7
8  print(square_lambda(5))  # 25
```

💡 Tip

当函数逻辑简单且只用一次时，用 `lambda` 很优雅。但如果逻辑复杂（含循环、多条件），还是老老实实写 `def`。

7. 高阶函数：把函数当作工具传来传去

定义：高阶函数就是 **接受函数作为参数** 或 **返回函数** 的函数。你已经见过 `sorted()` 的 `key` 参数。

常见搭档： `sorted()`、`filter()`、`map()`

7.1 `map(function, iterable)` —— 批量加工

把函数依次作用到每个元素上，返回一个新迭代器（通常用 `list()` 转换）。

```
python
1 | nums = [1, 2, 3, 4, 5]
2 | squares = list(map(lambda x: x**2, nums))
3 | print(squares)    # [1, 4, 9, 16, 25]
```

7.2 `filter(function, iterable)` —— 筛选

保留使函数返回 `True` 的元素。

```
python
1 | evens = list(filter(lambda x: x % 2 == 0, nums))
2 | print(evens)    # [2, 4]
```

7.3 `sorted(iterable, key=func)` —— 自定义排序

```
python
1 | students = [("张三", 85), ("李四", 72), ("王五", 90)]
2 | sorted_by_score = sorted(students, key=lambda s: s[1],
   | reverse=True)
3 | print(sorted_by_score) # [('王五', 90), ('张三', 85), ('李
   | 四', 72)]
```


8. 递归函数 —— 函数调用自己

8.1 什么是递归？

定义：一个函数在它的函数体内**调用自身**，称为递归。

生活类比：俄罗斯套娃，打开一个娃娃，里面还有一个更小的同样的娃娃。

递归通常用于解决可以分解为**相同子问题**的问题，例如：计算阶乘、斐波那契数列、遍历树形结构。

两个关键要素：

1. **递归出口（基例）：**终止条件，防止无限调用。
2. **递归步骤：**把大问题拆成小问题，调用自己。

8.2 递归的执行过程：压栈与弹栈

什么是压栈（Push）？

当函数调用自身时，当前函数的状态（参数、局部变量、返回地址）会被**压入**调用栈（Call Stack）顶部。栈就像一摞盘子，新调用的函数放在最上面。

什么是弹栈（Pop）？

当递归到达出口（终止条件）并返回时，最顶层的函数被**弹出**栈，返回值交给下一层函数继续计算，直到栈清空。



大白话：

- **压栈：**一层一层往里钻，先调用的函数在底下，后调用的在上面。
- **弹栈：**从最里面开始返回，一层一层往外退，每层算完就销毁。

8.3 递归示例：阶乘

数学定义：

$$n! = n \times (n-1) \times \dots \times 1, \text{ 且 } 1! = 1。$$



python

```
1  def factorial(n):
2      print(f"→ 压栈: 进入 factorial({n})")          # 模拟压栈
3      if n == 1:
4          print(f"← 弹栈: factorial(1) 返回 1")      # 递归出口
5          return 1
6      result = n * factorial(n - 1)
7      print(f"← 弹栈: factorial({n}) 返回 {result}")
8      return result
9
10 print("最终结果: ", factorial(3))
```

执行过程与栈变化（以 factorial(3) 为例）：

压栈过程

调用 factorial(3)

栈: [fact(3)]

fact(3) 调用 factorial(2)

栈: [fact(3), fact(2)]

fact(2) 调用 factorial(1)

栈: [fact(3), fact(2), fact(1)]

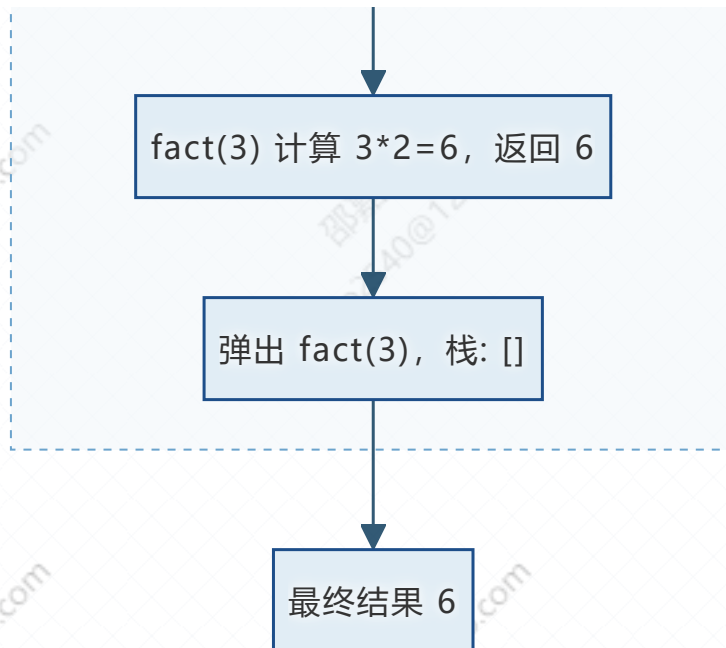
弹栈过程

fact(1) 到达出口, 返回 1

弹出 fact(1), 栈: [fact(3), fact(2)]

fact(2) 计算 $2 \times 1 = 2$, 返回 2

弹出 fact(2), 栈: [fact(3)]



详细文字说明：

步骤	动作	栈内容（底→顶）	说明
1	调用 <code>factorial(3)</code>	<code>[fact(3)]</code>	第一次压栈
2	<code>fact(3)</code> 调用 <code>factorial(2)</code>	<code>[fact(3), fact(2)]</code>	第二次压栈
3	<code>fact(2)</code> 调用 <code>factorial(1)</code>	<code>[fact(3), fact(2), fact(1)]</code>	第三次压栈，到达递归出口
4	<code>fact(1)</code> 返回 1	<code>[fact(3), fact(2)]</code>	弹栈，1 返回给 <code>fact(2)</code>
5	<code>fact(2)</code> 计算 <code>2*1=2</code> 并返回	<code>[fact(3)]</code>	弹栈，2 返回给 <code>fact(3)</code>
6	<code>fact(3)</code> 计算 <code>3*2=6</code> 并返回	<code>[]</code>	弹栈，得到最终结果 6



- 每一次递归调用都会**压栈**，占用内存。
- 递归深度过大会导致**栈溢出**（Python 默认约 1000 层）。
- 递归出口（终止条件）必须存在，否则会无限压栈，最终程序崩溃。

9. 函数的嵌套定义

9.1 什么是嵌套函数？

定义：在函数内部再定义另一个函数，称为**嵌套函数**（Nested Function）。内部函数只能在外部函数内部被调用，外部函数之外无法直接访问。

生活类比：就像一个公司，总经理办公室里有专属秘书，这个秘书只服务总经理，外面的人不能直接找秘书办事。

9.2 基本语法与调用

```
python
1  def outer():
2      print("外部函数开始")
3
4      def inner():
5          print("内部函数被调用")
6
7      inner()    # 内部函数只能在外部函数内调用
8      print("外部函数结束")
9
10 outer()
```

输出：

```
python
1  外部函数开始
2  内部函数被调用
3  外部函数结束
```



内部函数 `inner()` 只能在 `outer()` 内部使用。如果在函数外部尝试调用 `inner()`，会报 `NameError`。

9.3 嵌套函数的作用域

内部函数可以**读取**外部函数的变量（包括参数），但默认不能直接修改（需要 `nonlocal` 关键字，将在闭包中讲解）。

```
python
1  def outer(name):
2      def inner():
3          print(f"内部函数访问外层变量: {name}")
4      inner()
5
6  outer("小明")    # 输出: 内部函数访问外层变量: 小明
```

9.4 嵌套函数的用途

- **封装辅助逻辑**：把只在主函数内部使用的工具函数隐藏起来，不暴露给外部。
- **避免命名冲突**：内部函数的名称不会污染全局命名空间。
- **为闭包做准备**：当内部函数被返回并“记住”了外部变量时，就形成了闭包（下一节）。

10. 闭包 —— 函数“记住”了外部变量

10.1 什么是闭包？

闭包 (Closure) 是一个函数对象，它**记住了其外层作用域中的变量**，即使外层函数已经执行完毕，这些变量也不会被销毁。

闭包的三个必要条件：

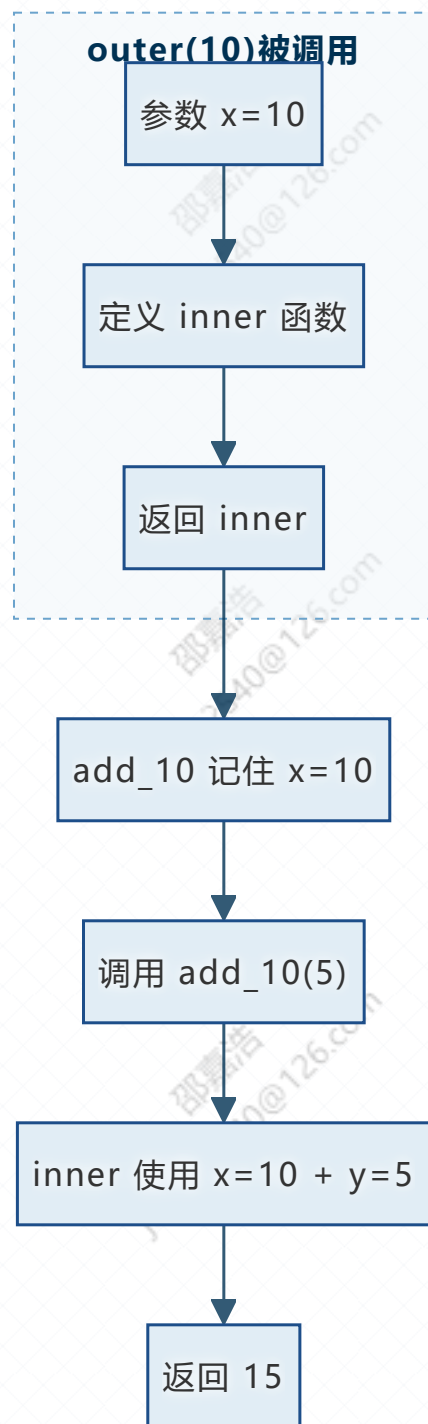
1. 存在**嵌套函数**（函数里面定义函数）。
2. 内部函数**使用了外部函数的变量**（包括参数）。
3. 外部函数**返回了内部函数**（而不是直接调用）。

大白话：闭包就像一个“背包”，函数即使离开了它的定义环境，还能“记住”当时环境中的变量。

10.2 闭包示例

```
python
1  def outer(x):
2      # x 是外部函数的变量
3      def inner(y):
4          return x + y    # 内部函数使用了外部变量 x
5      return inner        # 返回内部函数
6
7  # 创建闭包：add_10 记住了 x=10
8  add_10 = outer(10)
9  print(add_10(5))    # 15
```

流程图：闭包的形成



10.3 闭包的作用

- **数据隐藏**：通过闭包可以模拟“私有变量”，外部无法直接修改。
- **工厂函数**：根据不同的参数创建不同行为的函数（如上例的加法工厂）。
- **装饰器基础**（后续课程会讲）。

10.4 一个计数器闭包

```
python
1  def counter():
2      count = 0
3      def increment():
4          nonlocal count    # 声明使用外部函数的变量
5          count += 1
6          return count
7      return increment
8
9  c1 = counter()
10 print(c1()) # 1
11 print(c1()) # 2
12 print(c1()) # 3
13
14 c2 = counter()
15 print(c2()) # 1 (独立的计数)
```



- 如果内部函数要修改外部函数的变量，需要使用 `nonlocal` 关键字（类似 `global` 但用于嵌套作用域）。
- 闭包会延长外部函数中变量的生命周期。

10.5 闭包工厂函数示例

什么是工厂函数？

工厂函数 (Factory Function) 是指根据传入参数，生产（返回）不同功能的函数的函数。就像一家工厂，你告诉它“我要生产 2 倍的乘法器”，它就给你一个能翻倍的函数；你说“我要生产 3 倍的乘法器”，它就给你一个能变三倍的函数。

闭包如何实现工厂函数？

- 外部函数接收一个“配置参数”（如 `factor`）。

- 内部函数使用这个参数，形成定制化的行为。
- 外部函数返回内部函数，每次调用外部函数就会“生产”出一个新的、功能固定的函数。

python

```
1  def make_multiplier(factor):
2      """工厂函数：根据 factor 生产一个乘法器"""
3      def multiply(x):
4          return x * factor    # 内部函数记住了 factor
5      return multiply
6
7  # 生产不同的乘法器
8  double = make_multiplier(2)    # double 是一个闭包，记住了
   factor=2
9  triple = make_multiplier(3)    # triple 记住了 factor=3
10
11 print(double(5))    # 输出 10 (5 × 2)
12 print(triple(5))    # 输出 15 (5 × 3)
```

- `make_multiplier` 就是工厂函数。它不直接计算结果，而是“生产”出一个专门做乘法的小工具。
- 生产出来的 `double` 和 `triple` 各自记住了自己的 `factor`，互不干扰。
- 这种模式在实际开发中很常见：比如生成不同折扣的计算器、不同底数的幂函数等。

11. 课堂练习

基础函数练习

1. **最大公约数**：编写函数 `gcd(a, b)`，返回两个数的最大公约数（可使用辗转相除法）。
2. **判断质数**：编写函数 `is_prime(n)`，返回 `True` 或 `False`。

3. **可变参数平均值**: 编写函数 `average(*args)` , 返回传入的所有数值的平均值。如果没传任何参数, 返回 `0` 。

递归练习

1. **递归求和**: 编写递归函数 `sum_rec(n)` , 计算 `1 + 2 + ... + n` 的和。
2. **递归反转列表**: 编写递归函数 `reverse_list(lst)` , 返回反转后的新列表 (不使用切片或内置 `reverse`)。

闭包练习

1. **闭包计数器**: 编写闭包 `create_counter(start=0)` , 返回一个函数, 每次调用使计数器加1并返回当前值。要求可以独立创建多个计数器。
2. **闭包缓存计算**: 编写闭包 `make_adder(n)` , 返回一个函数 `adder(x)` , 返回 `x + n` 。